

INCIDENT RESPONSE

HANDBOOK

Build
Respond
Recover
Stronger



From First Alert to
Production Recovery



DETECT



TRIAGE



RESPOND



RECOVER



LEARN

WHAT YOU WILL MASTER

- ✓ Incident lifecycle
- ✓ Real-world troubleshooting
- ✓ Linux, Kubernetes and cloud diagnostics
- ✓ Mitigation and recovery strategies
- ✓ Communication and escalation
- ✓ Post-incident reviews
- ✓ Practical hands-on lab

REAL
INCIDENTS
REAL SKILLS
A MORE RESILIENT
YOU



PEOPLE

Work together



PROCESS

Follow a plan



TOOLS

Find the truth



RECOVERY

Restore service



IMPROVEMENT

Prevent next time

INCIDENTS HAPPEN.
PREPARED ENGINEERS MAKE THE DIFFERENCE.

Same
Incidents.
Stronger
Engineers.



@veriqta



veriqta



veriqta



veriqta



@veriqta



@veriqta

PRODUCTION INCIDENTS EXPLAINED, WHEN SOMETHING GOES WRONG

UNDERSTAND. RESPOND. RECOVER. LEARN.

1 WHAT AN INCIDENT IS



An incident is an unplanned interruption or degradation of a service that affects users or the normal operation of a system in production.

2 INCIDENT vs BUG vs PROBLEM vs SERVICE REQUEST

TERM	WHAT IT MEANS	EXAMPLE
Incident	Unplanned service interruption or degradation in production.	Website is down for users.
Bug	A fault in code or configuration.	Application error in specific feature.
Problem	The underlying cause of one or more incidents.	Database connection leaks causing repeated outages.
Service Request	A planned request for a change or resource.	Request for a new server or access.

3 WHAT MAKES AN INCIDENT PRODUCTION-IMPACTING

- Service is completely unavailable.
- Performance is significantly degraded (slow response times).
- High error rates.
- Core functionality is not working.
- Affects real users or critical internal systems.

4 AVAILABILITY, LATENCY, ERRORS AND DEGRADED SERVICE



AVAILABILITY
Service is up and reachable for users.



LATENCY
Service is slow to respond.



ERRORS
Requests are failing (e.g. 5xx errors).



DEGRADED SERVICE
Part of the service works, but not fully.

5 CUSTOMER IMPACT vs INTERNAL IMPACT



CUSTOMER IMPACT

- Affects external users
- Can impact revenue
- Higher urgency
- More communication required



INTERNAL IMPACT

- Affects internal teams or systems
- May not affect customers
- Still important
- Lower urgency than customer impact

6 WHY ENGINEERS NEED STRUCTURED INCIDENT RESPONSE

- ✓ Helps reduce confusion and chaos
- ✓ Ensures the right people are involved
- ✓ Improves communication
- ✓ Speeds up recovery
- ✓ Minimizes customer impact
- ✓ Helps identify the real root cause
- ✓ Prevents the same issue from happening again

7 INCIDENT RESPONSE MENTAL MODEL



8 JUNIOR ENGINEER TIP



Your first job is understanding the impact, not guessing the cause.

Get clear on:
What is affected? Who is affected?
How severe is it? What changed?

**CALM ENGINEERS SOLVE INCIDENTS.
PREPARED ENGINEERS PREVENT THEM.**

Learn
Build
Share
Grow



@veriqta



veriqta



veriqta



veriqta



@veriqta



@veriqta

THE INCIDENT LIFECYCLE FROM ALERT TO LEARNING

A structured approach helps teams respond faster, reduce impact, and prevent incidents from happening again.

1		DETECTION	An alert, monitoring signal, or user report indicates something is wrong. This is the first sign of an issue.
2		TRIAGE	Validate the alert, understand the scope, and assess customer impact. Determine if this is a real incident.
3		DECLARATION	Formally declare an incident (especially for SEV-1 or SEV-2). Open the incident channel and assign roles.
4		RESPONSE	Investigate, gather evidence, and work towards identifying the cause. Keep the team informed.
5		MITIGATION	Apply a fix or workaround to reduce or stop the immediate impact. Restore critical functionality.
6		RECOVERY	Bring the service back to normal or an acceptable level. Verify that the issue is resolved.
7		RESOLUTION	Confirm the incident is fully resolved. Close the incident and communicate the outcome.
8		POST-INCIDENT REVIEW	Analyze what happened, why it happened, and what can be improved. This is a blameless learning exercise.
9		FOLLOW-UP ACTIONS	Implement improvements such as better monitoring, automation, documentation, or architectural changes.

THE INCIDENT LIFECYCLE FLOW



JUNIOR ENGINEER TIP

Your first job is understanding the impact, not guessing the cause. Take time to confirm what is actually affected before taking action.



KEY TAKEAWAYS

- Follow a structured process
- Communicate clearly
- Focus on impact, not assumptions
- Recover the service, then learn and improve
- Every incident is an opportunity to build a more resilient system



@veriqta



veriqta



veriqta



veriqta



@veriqta



@veriqta

HOW PRODUCTION INCIDENTS ARE DETECTED

Incidents can be detected in many ways. The earlier we detect issues, the faster we can respond and reduce impact.

1 MONITORING ALERTS



- Triggered when a metric crosses a threshold (e.g. high CPU, high error rate).
- Examples: Prometheus, Grafana, Datadog, CloudWatch, Alertmanager.
- Helps detect issues before users report them.

2 LOGS



- Show detailed events and error messages.
- Help identify what is happening.
- Examples: Application logs, system logs, container logs.
- Useful for understanding the cause and timeline.

3 METRICS



- Provide numerical data about system health.
- Examples: CPU, memory, latency, request rate, error rate.
- Help identify unusual patterns or spikes.
- Used for alerting and performance analysis.

4 DISTRIBUTED TRACES



- Track a request across multiple services.
- Show where latency or errors occur.
- Examples: Jaeger, Zipkin, AWS X-Ray, OpenTelemetry.
- Useful in microservices and complex systems.

5 SYNTHETIC MONITORING



- Uses automated tests to simulate user activity.
- Checks if your application is reachable and working.
- Examples: Pingdom, UptimeRobot, Datadog Synthetics.
- Helps detect issues from a user's perspective.

6 HEALTH CHECKS



- Regular checks to confirm a service is healthy.
- Examples: Kubernetes readiness and liveness probes, load balancer health checks.
- Detects failures quickly.
- Helps with automatic recovery in some cases.

7 CUSTOMER REPORTS



- Users report issues through support channels (email, chat, helpdesk).
- Often the first sign of an incident.
- Helps confirm real user impact.
- Should always be taken seriously.

8 INTERNAL REPORTS



- Team members notice issues (e.g. slower tools, failed jobs).
- Reports from other teams (e.g. Dev, QA, Security).
- Helps detect issues that monitoring might miss.
- Encourages a culture of speaking up.

9 SLO BREACHES



- Occurs when a service does not meet its Service Level Objective (SLO).
- Examples: 99.9% availability, 200ms latency.
- Indicates the service is not meeting its reliability goals.
- Often used to trigger incident response.

10 WHY AN ALERT IS EVIDENCE, NOT NECESSARILY THE ROOT CAUSE



- An alert tells you that something is wrong.
- It shows a symptom, not the cause.
- Multiple alerts may point to the same issue.
- You still need to investigate, gather evidence, and find the real root cause.
- Always validate and explore before making changes.

EXAMPLE: INCIDENT DETECTION FLOW



JUNIOR ENGINEER TIP

An alert is the beginning of the investigation, not the answer. Focus on understanding the impact and gathering evidence.

KEY TAKEAWAY

Incidents can be detected from many sources. Always validate alerts, understand the impact, and investigate before taking action.

Incidents
Happen.
Better
Engineers
Recover.

TRIAGE, WHAT IS ACTUALLY BROKEN?

Triage is the first step to understand the real issue, confirm the impact, and gather enough information to take the right action.

1



CONFIRMING THE ALERT

- Verify the alert is real and not a false positive.
- Check if the issue is still happening.
- Look at metrics, logs or the actual service.

2



IDENTIFYING AFFECTED SERVICES

- Find out which services, environments or regions are affected.
- Check dependencies (databases, APIs, external services).
- Understand the blast radius.

3



DETERMINING CUSTOMER IMPACT

- Is the service completely down or degraded?
- How many users are affected?
- Is there revenue, data or reputation impact?

4



CHECKING WHEN THE PROBLEM STARTED

- Identify the exact time the issue began.
- Compare with monitoring graphs, logs and deployments.
- Look for patterns or recurring issues.

5



LOOKING FOR RECENT CHANGES

- Check recent deployments, configuration changes, infrastructure changes or scaling events.
- Review change logs, CI/CD pipelines and release notes.
- Ask: What changed before the issue started?

6



ESTABLISHING SCOPE

- Is it one pod, one service, one cluster or multiple regions?
- Is it environment specific (dev, staging, production)?
- Identify what is not affected (this helps narrow the cause).

7



COMPARING HEALTHY AND UNHEALTHY COMPONENTS

- Compare metrics, logs and configuration between working and non-working components.
- Look for differences in versions, settings or resource usage.
- Identify what is unique to the failing component.

8



INITIAL EVIDENCE COLLECTION

- Gather logs, metrics, traces and error messages.
- Take screenshots or record outputs.
- Document what you see (time, findings, commands).
- This helps the team and the post-incident review.

9

TRIAGE QUESTIONS EVERY JUNIOR ENGINEER SHOULD ASK

1

Is the alert real or a false positive?

2

What service is affected?

3

Is the service completely down or just degraded?

4

Who is affected (customers or internal users)?

5

When did the issue start?

6

What changed recently?

7

Are there any related alerts?

8

What do the logs, metrics or traces show?

9

Are dependencies (databases, APIs, external services) healthy?

10

Can I reproduce the issue?

11

Is this happening in other environments or regions?

12

What is working as expected?

13

Do I have enough information, or do I need to escalate?



JUNIOR ENGINEER TIP

Triage is about understanding the impact and gathering facts, not guessing the cause. Take your time, ask the right questions, and avoid making changes until you have enough information.



REMEMBER

- ✓ Confirm the issue.
- ✓ Understand the impact.
- ✓ Gather evidence.
- ✓ Communicate clearly.

GOOD TRIAGE LEADS TO FASTER RECOVERY
AND LESS CUSTOMER IMPACT.

Same
Incidents.
Stronger
Engineers.

SEVERITY LEVELS AND INCIDENT CLASSIFICATION

Severity helps teams prioritize, respond appropriately, and minimize impact.
Use clear criteria, not emotions, to determine severity.

1 SEVERITY LEVELS

SEVERITY	NAME	DESCRIPTION	EXAMPLE SCENARIOS
SEV-1	Critical	- Major outage or severe degradation. - Large customer impact. - Immediate response required.	- Complete service outage. - Core functionality down. - Widespread customer impact. - Security breach in production.
SEV-2	High	- Significant impact, but service partially available. - Limited customer impact compared to SEV-1. - High priority response.	- Key feature unavailable. - High error rates or latency. - Degraded performance for many users.
SEV-3	Medium	- Moderate impact. - Non-critical functionality affected. - Normal response time.	- Minor feature not working. - Intermittent errors. - Limited customer impact.
SEV-4	Low	- Minimal impact. - No significant customer impact. - Can be addressed during normal working hours.	- Cosmetic issues. - Single user affected. - Informational alerts.

2 EXAMPLE SEVERITY CRITERIA

	Customer impact How many users are affected?
	Scope Single service or multiple services?
	Duration Is it ongoing or short-lived?
	Security implications Is there a security risk?
	Data integrity Is data at risk or lost?
	Revenue or business impact Does this affect revenue or key business goals?

3 ESCALATION THRESHOLDS

	SEV-1	- Page on-call immediately - Notify leadership - War room (if required) - Frequent updates
	SEV-2	- Page on-call - Notify service owner - Regular updates - Escalate if impact increases
	SEV-3	- Create incident ticket - Notify relevant team - Monitor and update - Escalate if the situation worsens
	SEV-4	- Create ticket - Handle in normal workflow - No immediate paging - Monitor for changes

4 WHY SEVERITY MAY CHANGE DURING AN INCIDENT



- More users are affected than initially thought.
- The impact on business or revenue becomes clearer.
- A security risk is discovered.
- The issue is lasting longer than expected.
- New information shows a wider scope.
- The situation improves after mitigation.
- Better understanding of the true impact.

5 KEY TAKEAWAYS



- Use clear and consistent criteria.
- Focus on customer impact and business risk.
- Reassess severity as new information becomes available.
- Communicate changes clearly to all stakeholders.
- Correct severity leads to faster and more effective response.



JUNIOR ENGINEER TIP

Severity is not about assumptions. Use facts, ask questions, and be ready to adjust the severity as you learn more during the incident.

RIGHT SEVERITY. FASTER RESPONSE.
LOWER IMPACT. STRONGER SYSTEMS.

Same
Incidents.
Stronger
Engineers.



@veriqta



veriqta



veriqta



veriqta



@veriqta



@veriqta

WHO DOES WHAT DURING AN INCIDENT?

Clear roles, clear communication, faster recovery. Everyone has a part to play. The right people, doing the right things, reduces chaos and minimizes impact.

1 INCIDENT COMMANDER



- Owns the incident end to end.
- Makes key decisions.
- Coordinates the response.
- Ensures clear communication.
- Decides when the incident is resolved.

Focus: People, process and overall response.

2 TECHNICAL LEAD



- Leads the technical investigation.
- Guides troubleshooting and mitigation.
- Works with subject matter experts.
- Recommends technical solutions.
- Updates the Incident Commander.

Focus: Technical direction and problem solving.

3 OPERATIONS RESPONDERS



- Investigate and execute runbooks.
- Perform diagnostics and implement fixes.
- Monitor system behavior.
- Provide updates on their progress.

Focus: Hands-on response and execution.

4 COMMUNICATIONS LEAD



- Manages internal and external communications.
- Prepares status updates.
- Ensures consistent and accurate messaging.
- Works with leadership and customer support.

Focus: Clear, timely and factual communication.

5 SUBJECT MATTER EXPERTS (SMEs)



- Provide deep knowledge in specific areas (e.g. databases, networking, security).
- Help with complex issues.
- Advise on possible solutions.
- Join as needed.

Focus: Specialized expertise.

6 SCRIBE



- Records the incident timeline.
- Logs key decisions, actions and findings.
- Tracks who did what and when.
- Helps with the post-incident review.

Focus: Accurate documentation.

7 SERVICE OWNER



- Understands the service and its dependencies.
- Provides business context.
- Helps assess customer impact.
- Approves significant changes (if required).

Focus: Service context and business impact.

8 ESCALATION CONTACTS



- Senior engineers, managers or leadership.
- Brought in when additional support is needed.
- Help remove blockers.
- Make higher-level decisions.
- May include security, legal or external vendors.

Focus: Support, escalation and risk management.

9 CLEAR OWNERSHIP



- Everyone should know their role.
- Avoid confusion and overlapping work.
- Use an incident channel (e.g. Slack, Teams).
- Follow the established incident process.
- Clear ownership leads to faster, safer recovery.

10 WHY TOO MANY ENGINEERS MAKING CHANGES CAN MAKE AN INCIDENT WORSE



- Can cause conflicting changes.
- Makes it harder to identify what fixed or broke the issue.
- Increases the risk of additional failures.
- Creates confusion and noise.
- Slows down recovery.
- Always coordinate changes through the Incident Commander or Technical Lead.



KEY TAKEAWAYS

- Right people. Right roles. Better outcomes.
- Follow the incident process.
- Communicate clearly.
- Avoid unnecessary changes.
- Work together to restore the service safely.



JUNIOR ENGINEER TIP

Know your role, ask questions, and communicate. If you are not sure, do not make changes. It is better to ask than to make the situation worse.

INCIDENTS ARE TEAM EFFORTS.
CLEAR ROLES TURN CHAOS INTO RECOVERY.

Better
Engineers
Brighter
Systems



@veriqta



veriqta



veriqta



veriqta



@veriqta



@veriqta



THE FIRST 15 MINUTES OF A PRODUCTION INCIDENT

The first 15 minutes set the tone for the entire incident. Stay calm, focus on facts, and take the right steps in the right order.

1 ACKNOWLEDGE THE ALERT



- Confirm you have seen the alert.
- Stop what you are doing and focus.
- Let the team know you are investigating.

"Acknowledged and looking into this now."

2 CONFIRM IMPACT



- Verify if the issue is real and not a false positive.
- Check if customers are affected.
- Look at metrics, logs or the actual service.

"Is this really impacting users?"

3 CHECK SERVICE HEALTH



- Check dashboards and health checks.
- Verify key services and dependencies.
- Look for error rates, latency and availability.

"What is unhealthy and how widespread is it?"

4 DETERMINE SEVERITY



- Assess the customer impact.
- Use your organization's severity criteria (SEV-1, SEV-2, SEV-3, SEV-4).
- Escalate quickly if it looks major.

"How severe is this right now?"

5 DECLARE THE INCIDENT WHEN APPROPRIATE



- Declare an incident for SEV-1 or SEV-2 (or as per your organization's policy).
- This starts the formal response process.
- Ensures the right people are notified.

"We are declaring an incident."

6 OPEN THE INCIDENT CHANNEL



- Create or use the designated incident channel (e.g. Slack, Teams).
- Include the right stakeholders.
- Keep communication centralized.

"All incident communication goes here."

7 ASSIGN OWNERSHIP



- Confirm the Incident Commander (IC).
- Assign responders and key roles.
- Make sure everyone knows their responsibility.

"Who is leading and who is doing what?"

8 CHECK RECENT DEPLOYMENTS AND INFRASTRUCTURE CHANGES



- Look for recent application deployments.
- Check infrastructure, configuration or scaling changes.
- Identify anything that might be related.

"What changed recently?"

9 BEGIN A TIMELINE



- Note the time of the first alert.
- Record key events, actions and decisions.
- This helps with communication and the post-incident review.

"Start the timeline early."

10 COMMUNICATE INITIAL STATUS



- Share what is known.
- Share what is not yet known.
- Provide the current impact and next update time.
- Keep stakeholders informed.

"Be clear, factual and concise."

11 AVOID PREMATURE ROOT-CAUSE CLAIMS



- Do not guess the cause.
- Focus on understanding the impact first.
- Let the evidence guide the investigation.
- Premature conclusions can lead to the wrong fixes.

"It's okay to say: 'We are investigating.'"

KEY TAKEAWAYS



- ✓ Act quickly and calmly.
- ✓ Focus on impact, not assumptions.
- ✓ Communicate clearly.
- ✓ Get the right people involved.
- ✓ Follow a structured process.
- ✓ The first 15 minutes can reduce the impact and speed up recovery.



JUNIOR ENGINEER TIP

Your job is not to fix everything in the first 15 minutes. Your job is to understand the impact, gather evidence, and help the team respond.

CALM ENGINEERS
SOLVE INCIDENTS FASTER.

OBSERVABILITY DURING INCIDENT RESPONSE

Metrics. Logs. Traces. Dashboards. Answers.

1 METRICS



Metrics tell you what changed.

- Numerical data over time
- Shows patterns and trends
- Helps you detect anomalies quickly

Example: CPU usage suddenly increased from 20% to 90%

2 LOGS



Logs provide event details.

- Detailed, time-stamped events
- Shows what happened
- Helps you find errors and root causes

Example: Application error: "connection timeout to database"

3 TRACES



Traces follow requests.

- Tracks a request across services
- Shows where the delay or error occurred
- Essential for microservices

Example: Request slow in service A, timeout in service B

4 DASHBOARDS



Dashboards provide system context.

- Combines metrics, logs and traces
- Shows overall health at a glance
- Helps you see the bigger picture

Example: Shows high error rate, high latency and affected services

5 THE FOUR GOLDEN SIGNALS

These key metrics help you quickly understand the health of a service.



LATENCY

How long requests take to complete.



TRAFFIC

How many requests you are receiving.



ERRORS

How many requests fail.



SATURATION

How full your resources are (CPU, memory, disk, etc).

6 CORRELATION vs CAUSATION



CORRELATION

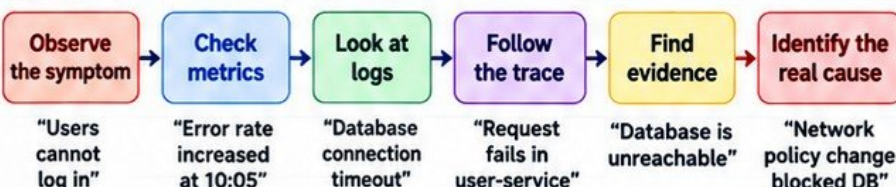
- Two things change at the same time.
- Does NOT mean one caused the other.
- Example: CPU is high and error rate is high. They might be related, but CPU might not be the cause.



CAUSATION

- One thing directly causes another.
- You confirm it with evidence.
- Example: A memory leak causes high memory usage, which causes Pods to restart.

7 MOVING FROM SYMPTOM TO EVIDENCE



8 JUNIOR ENGINEER TIP



Do not rely on a single signal. Use metrics, logs, traces and dashboards together. They give you the full picture and help you move from guessing to evidence-based investigation.

OBSERVABILITY TURNS INCIDENTS FROM GUESSWORK INTO FACTS. FIND THE EVIDENCE. FIX THE REAL PROBLEM.



@veriqta



veriqta



veriqta



veriqta



@veriqta



@veriqta

Same
Engineers.
Bigger
Impact.

ESSENTIAL LINUX AND INFRASTRUCTURE DIAGNOSTICS

Find the problem. Get the evidence. Fix with confidence.

COMMAND	WHAT IT CONFIRMS	EXAMPLE	WHEN TO USE IT
 uptime	System uptime, load average	uptime	Check if the server is under high load or recently rebooted
 top	Real-time CPU, memory, process usage	top	Identify processes using high resources
 free -h	Memory usage (RAM)	free -h	Check available memory and detect memory issues
 df -h	Disk space usage	df -h	Check disk space and find full partitions
 ps	Running processes	ps aux	Find running processes and troubleshoot applications
 ss	Open ports and network connections	ss -tulnp	Check if a service is listening on the expected port
 systemctl status	Service status	systemctl status nginx	Check if a service is running and view its current status
 journalctl	Service logs (systemd)	journalctl -u nginx -n 50	View recent logs for a service and identify errors
 curl	HTTP/API connectivity	curl -I https://example.com	Test if an endpoint is reachable and check HTTP response
 dig	DNS resolution	dig example.com	Check DNS resolution and troubleshoot DNS issues



KEY AREAS THESE COMMANDS HELP YOU CHECK

- CPU issues
- Memory issues
- Disk space issues
- Process problems
- Port and network issues
- Service failures
- HTTP and API issues
- DNS resolution problems



SAFE PRODUCTION DIAGNOSTICS

- Start with read-only commands
- Do not make unnecessary changes
- Be careful with kill, rm, or configuration edits
- Confirm the impact before taking action
- Collect evidence (logs, outputs)
- Communicate what you are doing
- If unsure, ask a teammate
- Document your findings



IMPORTANT REMINDER

- These commands help you find the problem.
- Always understand the output.
- Do not guess.
- One command can give you a clue, but you may need multiple commands to find the real cause.
- When in doubt, observe, gather evidence, and ask for help.

OBSERVE. INVESTIGATE. RESOLVE.
THAT'S HOW RELIABLE SYSTEMS ARE BUILT.



@veriqta



veriqta



veriqta



veriqta



@veriqta



@veriqta

Better Engineers
Build Better Systems

KUBERNETES INCIDENT TROUBLESHOOTING

Find the issue. Understand the cause. Restore the service.

1 ESSENTIAL KUBECTL COMMANDS

COMMAND	WHAT IT DOES	EXAMPLE
<code>kubectl get pods</code>	List all pods in the current namespace	<code>kubectl get pods</code>
<code>kubectl describe pod</code>	Show detailed information about a specific pod	<code>kubectl describe pod <pod-name></code>
<code>kubectl logs</code>	View logs from a pod	<code>kubectl logs <pod-name></code>
<code>kubectl get events</code>	Show recent events in the namespace	<code>kubectl get events</code>
<code>kubectl get deployment</code>	List deployments	<code>kubectl get deployments</code>
<code>kubectl get service</code>	List services	<code>kubectl get services</code>
<code>kubectl get endpoints</code>	Show endpoints behind a service	<code>kubectl get endpoints <svc-name></code>

2 COMMON POD ERRORS AND WHAT THEY MEAN

STATUS	WHAT IT MEANS	WHAT TO CHECK
CrashLoopBackOff	Container is repeatedly crashing	Check logs, app config, dependency, env vars
ImagePullBackOff	Cannot pull the container image	Check image name, registry, credentials, network
Pending	Pod is waiting for resources or scheduling	Check nodes, resources, PVC, node selectors, taints, quotas
Failed probes	Liveness or readiness probe is failing	Check probe config, application health, startup time
OOMKilled	Container was killed due to out of memory	Check memory limits, application memory usage, tune resources

3 TROUBLESHOOTING WORKFLOW



4 APPLICATION vs KUBERNETES FAILURES

LIKELY APPLICATION FAILURE

- Pod is running but app is unhealthy
- Application errors in logs
- Failed readiness probe (app issue)
- High error rate from the app
- Database or external dependency errors
- Application misconfiguration
- Bug in the code

LIKELY KUBERNETES FAILURE

- Pod stuck in Pending
- ImagePullBackOff
- CrashLoopBackOff (startup issue)
- Node not ready
- Insufficient resources
- PVC or storage issues
- Service or endpoint misconfiguration
- Network or CNI issues
- DNS resolution problems

5 QUICK CHECKS FOR COMMON ISSUES

ISSUE	WHAT TO CHECK
Pod not starting	describe pod, logs, events, image, resources
Service not working	get service, get endpoints, check pod labels/selectors
Cannot pull image	image name, registry access, secrets, network
Pod pending	node resources, taints/tolerations, PVC, quotas
Probe failing	probe config, app readiness, logs
High memory usage	<code>kubectl top pod</code> , memory limits, application tuning
DNS not resolving	check CoreDNS pods, DNS config, network policies

6 SAFE PRODUCTION DIAGNOSTICS

- Start with read-only commands.
- Do not delete or restart resources unless necessary.
- Understand the impact before making changes.
- Check multiple signals (pods, logs, events, services).
- Document what you find.
- Communicate with your team.
- If unsure, ask for help.
- Avoid making multiple changes at the same time.
- Use a separate terminal for investigation.
- Follow your organization's incident runbooks.

OBSERVE. INVESTIGATE. SOLVE. KEEP THE SERVICE RUNNING.

Same
Incidents.
Stronger
Engineers.

NETWORK AND DNS INCIDENTS

Find the issue. Follow the path. Restore the service.

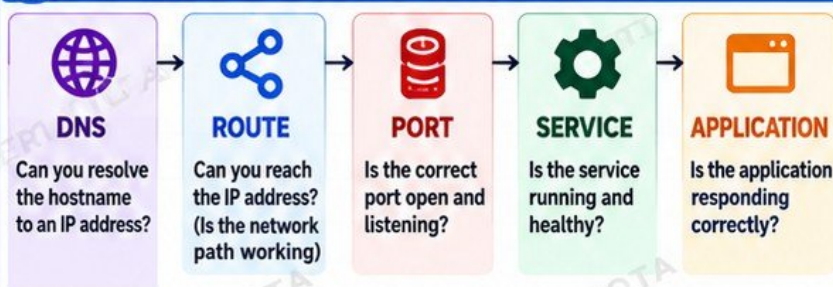
1 COMMON NETWORK AND DNS INCIDENTS

1	Connection refused	The target host is reachable but nothing is listening on the port.
2	Connection timeout	No response from the target host.
3	DNS resolution failure	The hostname cannot be resolved to an IP address.
4	Wrong port	Connecting to the wrong port or service.
5	Unreachable dependency	A dependent service is down or not reachable.
6	Load balancer problems	Health checks failing or traffic not reaching targets.
7	Firewall and security rules	Traffic is blocked by firewall, security group or network policy.
8	Service discovery	The application cannot find the correct service endpoint.

2 USEFUL COMMANDS

COMMAND	WHAT IT DOES	EXAMPLE
curl	Test HTTP/HTTPS connectivity	curl -v https://example.com
dig	Query DNS records	dig example.com
nslookup	Check DNS resolution	nslookup example.com
ss	Check open ports and connections	ss -tulnp
ping	Test basic network connectivity	ping example.com
tracert	Trace the network path	tracert example.com
telnet	Test connectivity to a port	telnet example.com 80
nc	Test a port using netcat	nc -zv example.com 80

3 NETWORK TROUBLESHOOTING PATH



4 EXAMPLE: TROUBLESHOOTING FLOW

- 1 Check DNS resolution
dig example.com
- 2 Check network reachability
ping <ip-address> or traceroute example.com
- 3 Check if the port is open
ss -tulnp or nc -zv example.com 80
- 4 Check the service
curl -v http://example.com
- 5 Check the application
Verify the response and logs

5 COMMON ERROR MESSAGES AND WHAT THEY MEAN

ERROR MESSAGE	LIKELY CAUSE
Connection refused	Service is not running or wrong port
Connection timed out	Host unreachable or firewall blocking traffic
Could not resolve host	DNS resolution failure
No route to host	Network routing issue
TLS handshake failed	SSL/TLS or certificate problem
502 / 503 from load balancer	Backend service is unhealthy
Name or service not known	DNS or service discovery issue



IMPORTANT REMINDER

- A network issue can look like an application issue.
- Always verify DNS, network path and port connectivity first.
- Many "application outages" are actually network or DNS problems.
- Check both sides: client and server.

6 QUICK TEST EXAMPLES

```
curl -v https://example.com # Test HTTP
dig example.com             # DNS lookup
nslookup example.com        # DNS lookup
ss -tulnp                   # Check open ports
ping example.com             # Check connectivity
tracert example.com          # Trace network path
nc -zv example.com 80        # Test port 80
curl -I http://example.com   # Check HTTP headers
```

DIAGNOSE SYSTEMATICALLY.
FIX CONFIDENTLY.
KEEP SERVICES RUNNING.

7 JUNIOR ENGINEER TIPS

- ✓ Follow the troubleshooting path (DNS → Route → Port → Service → Application).
- ✓ Check from the same network as the application.
- ✓ Verify security group, firewall and network policies.
- ✓ Do not assume it is a DNS problem.
- ✓ Use multiple tools to confirm your findings.
- ✓ Document what you check and the results.
- ✓ If unsure, ask a senior engineer.
- ✓ Keep changes minimal during an incident.

Same
Incidents.
Stronger
Engineers.

DEPLOYMENT AND CHANGE-RELATED INCIDENTS

Recent changes are one of the biggest causes of production incidents.

1 WHY RECENT CHANGES MATTER

- Most production incidents are caused by recent changes.
- Changes can introduce new bugs, misconfigurations or unexpected behavior.
- The closer the change to the incident time, the higher the probability it is related.
- Checking recent changes helps you focus your investigation and save time.
- It does not always mean the change is the root cause, but it is the first place to look.



2 COMMON CHANGE-RELATED INCIDENTS

	Bad application releases	New version contains a bug or breaking change.
	Configuration changes	Incorrect or incompatible configuration.
	Infrastructure changes	Changes to compute, network, storage or scaling.
	Dependency upgrades	New version causes compatibility issues.
	Feature flags	Incorrect flag state or unintended feature exposure.
	Database migrations	Migration failure or performance impact.

3 CHECK RECENT CHANGES

- Application deployments (what version?)
- Configuration changes
- Infrastructure changes (cloud, network, scaling)
- Dependency upgrades (libraries, images, plugins)
- Feature flag changes
- Database schema changes or migrations
- Changes in monitoring, alerts or logging
- Security rule changes (firewall, IAM, etc.)
- Check change management system, Git commits, CI/CD pipeline, and deployment history



4 COMPARE BEFORE AND AFTER

BEFORE (KNOWN GOOD STATE)

- Previous working version
- Previous configuration
- Previous infrastructure state
- Previous dependency versions
- Previous feature flag state
- Previous database schema
- System metrics and logs before the change

AFTER (CURRENT STATE)

- New version
- New configuration
- New infrastructure state
- New dependency versions
- New feature flag state
- New database schema
- Current metrics and logs
- Identify what is different

The difference often reveals the cause.

5 ROLL FORWARD vs ROLLBACK

ROLL FORWARD (Fix and deploy a new version)

- Used when you can quickly fix the issue
- Preferred when rollback is risky (e.g. database changes)
- Keeps moving forward
- Requires confidence and testing

ROLLBACK (Return to a previous version)

- Used when the new version is causing the incident
- Faster way to restore service
- Works only if the previous version is stable
- May not be possible if there are database or schema changes

6 THE MOST VALUABLE INCIDENT QUESTION

“What changed?”

- This question quickly narrows down the investigation.
- It helps you find the real cause instead of guessing.
- Always check deployments, configurations, infrastructure, and dependencies.
- Even if the change is not the root cause, it often provides critical clues.
- Asking “What changed?” early is a key incident response skill.

7 REAL-WORLD EXAMPLE

A new application version is deployed. Error rate increases, pods start crashing. Checking the deployment history shows a recent change. Rolling back to the previous version restores service.

8 JUNIOR ENGINEER TIP



Always check what changed before going deep into complex debugging. It can save hours and help you find the real issue faster.

CHANGES BUILD FEATURES. INVESTIGATION KEEPS THEM RUNNING.

Same
Incidents.
Stronger
Engineers.



@veriqta



veriqta



veriqta



veriqta



@veriqta



@veriqta

MITIGATION vs ROOT CAUSE

Restore the service. Find the real cause. Build a stronger system.

1 WHAT MITIGATION MEANS



Mitigation is a temporary or immediate action to reduce or remove the impact of an incident.

- Focuses on getting the service working again.
- Does not always fix the underlying cause.
- Buys time for a full investigation.
- Can be a short-term or long-term measure.

2 WHAT ROOT CAUSE MEANS



Root cause is the underlying reason why the incident happened.

- Identifies the real problem.
- Can involve multiple contributing factors.
- Requires investigation, analysis and evidence.
- Helps prevent the incident from happening again.

3 WHY RECOVERY MAY COME BEFORE FULL DIAGNOSIS



In a production incident, the priority is to restore service and reduce customer impact.

- Users should not stay down while you investigate.
- You can mitigate first and continue diagnosis in parallel.
- Full root cause analysis may take hours or days.
- Restoring the service quickly builds trust and reduces business impact.

4 COMMON MITIGATION ACTIONS

ACTION	WHAT IT DOES
 Rollback	Revert to a previous working version.
 Failover	Switch traffic to a healthy standby system.
 Scaling	Increase resources to handle load (pods, instances, etc.).
 Disabling a feature	Turn off a problematic feature or service.
 Removing bad traffic	Block or filter malicious or faulty traffic.
 Restoring configuration	Revert to a known good configuration.
 Restarting safely	Restart a service, pod or node in a controlled way.

5 TEMPORARY MITIGATION vs PERMANENT CORRECTION

TEMPORARY MITIGATION



- Focuses on immediate recovery
- May not fix the underlying cause
- Used to reduce impact quickly
- Examples: rollback, scale, disable feature, restart
- Can be removed later
- Buys time for proper analysis

PERMANENT CORRECTION



- Addresses the root cause
- Prevents the incident from happening again
- Requires full investigation
- Examples: code fix, configuration change, infrastructure improvement
- Often implemented after a post-incident review
- Improves system reliability

“ Mitigation gets you back online.
Root cause keeps you there. ”

6 KEY TAKEAWAYS

- ✓ Mitigation and root cause are different but both are important.
- ✓ Recovery may come before full diagnosis.
- ✓ Use the right mitigation for the situation.
- ✓ Never keep users down just to finish an investigation.
- ✓ Always follow up with a root cause analysis.
- ✓ Implement permanent fixes to prevent recurrence.

7 JUNIOR ENGINEER TIP



- Focus on restoring the service and reducing customer impact first.
- Communicate clearly about what you are doing.
- Do not chase the root cause at the expense of recovery.
- After the service is stable, help with the investigation and learn from it.
- Every incident is a chance to make the system stronger.

RESTORE. INVESTIGATE. LEARN. BUILD A STRONGER SYSTEM.

Same
Incidents.
Stronger
Engineers.



@veriqta



veriqta



veriqta



veriqta



@veriqta



@veriqta

SAFE PRODUCTION CHANGES DURING AN INCIDENT

**DON'T
MAKE A BAD
SITUATION
WORSE.**

Move carefully. Be deliberate. Reduce risk. Restore service.

1



CHANGE ONE THING AT A TIME

Make a single, controlled change so you know what fixed the issue or caused a new problem.

2



STATE YOUR HYPOTHESIS

Clearly state what you think is causing the issue and how your change will help. This keeps the team aligned.

3



KNOW THE EXPECTED RESULT

Be clear on what should happen if the change works. This helps you quickly confirm success or failure.

4



RECORD THE CHANGE

Document what you changed, when you changed it, why you changed it, and the result. This is crucial for the timeline and post-incident review.

5



UNDERSTAND BLAST RADIUS

Know what the change can affect. Consider dependent services, users, regions, and data. Avoid making wide changes during an incident.

6



GET PEER REVIEW FOR RISKY ACTIONS

For high-risk changes, get a second engineer to review the plan before execution, even during an incident.

7



PRESERVE EVIDENCE WHERE POSSIBLE

Keep logs, metrics, and system state. Avoid deleting data or restarting components unless necessary. You may need this for root cause analysis.

8



AVOID DESTRUCTIVE COMMANDS

Be extremely careful with commands that delete, overwrite, or permanently change data. Double-check before running anything in production.

9



PREPARE ROLLBACK

Have a rollback plan before making the change. Know how to revert if the change does not work or causes more issues.

10



VERIFY EVERY CHANGE

After the change, confirm the expected result. Check metrics, logs, service health, and user impact. Do not assume it worked.

11



SECURITY AND LEAST PRIVILEGE DURING EMERGENCIES

Even during an incident, follow security best practices. Use the minimum required access. Avoid using root or overly permissive credentials. Protect sensitive data and credentials.



GOOD CHANGE MINDSET

"Be intentional, stay calm, and think about the impact."



COMMON MISTAKE

"Making multiple changes at once and not knowing what fixed the issue."



REMEMBER

"Speed is important, but a safe and correct fix is what actually restores and keeps the service healthy."

SAFE CHANGES TODAY.
A MORE RELIABLE TOMORROW.



@veriqta



veriqta



veriqta



veriqta



@veriqta



@veriqta

Learn
Build
Grow
with
VERIQTA

INCIDENT COMMUNICATION THAT ENGINEERS NEED

Clear. Accurate. Timely. Build trust.

GOOD
COMMUNICATION
REDUCES
CONFUSION
AND BUILDS
TRUST.

1 INTERNAL INCIDENT UPDATES



- For engineers, on-call team, and internal stakeholders.
- Share technical details, findings, and actions.
- Keep updates short and factual.
- Use the incident channel (Slack, Teams, etc.).
- Keep a timeline of key events.

2 STAKEHOLDER COMMUNICATION



- For management, product, security, and business teams.
- Focus on impact, actions, and next steps.
- Avoid unnecessary technical jargon.
- Be consistent and provide regular updates.
- Escalate when impact or severity changes.

3 CUSTOMER-FACING UPDATES



- For end users and customers.
- Be clear, simple, and non-technical where possible.
- Acknowledge the issue.
- Share the impact and what you are doing.
- Provide regular updates.
- Only announce recovery after full verification.

4 WHAT TO INCLUDE IN EVERY UPDATE

WHAT IS KNOWN	Facts that are confirmed.
WHAT IS UNKNOWN	Things you are still investigating.
CURRENT IMPACT	How the issue affects users or services.
ACTIONS UNDERWAY	What the team is doing now.
NEXT UPDATE TIME	When the next update will be shared.

5 STATUS UPDATE TEMPLATE



[Incident Name] – Update #X
Time: YYYY-MM-DD HH:MM (UTC)
What is known: <brief summary>
What is unknown: <brief summary>
Current impact: <customer or service impact>
Actions underway: <what the team is doing>
Next update: <time of next update>

6 GOOD EXAMPLE (INTERNAL)



Update #2 – 2026-07-16 14:30 UTC
What is known: API returning 500 errors since 14:10 UTC.
What is unknown: Exact root cause.
Current impact: 60% of requests failing.
Actions underway: Checking recent deployment and database connections.
Next update: 14:45 UTC.

7 GOOD EXAMPLE (CUSTOMER)



We are currently experiencing issues with our service. Some users may see errors when trying to access the platform.
Our team is investigating the issue and working on a fix. We will provide another update in 30 minutes.
Thank you for your patience.

8 AVOID THESE MISTAKES



- Speculating about the root cause.
- Giving false recovery announcements.
- Using vague updates like “investigating” for hours.
- Sharing unconfirmed information.
- Blaming individuals or teams.
- Overly technical language in customer updates.

9 KEY TAKEAWAYS



- Be clear, concise, and factual.
- Share what is known and unknown.
- Set expectations with a next update time.
- Avoid speculation.
- Keep communication consistent.
- Only announce recovery after verification.
- Good communication builds trust and reduces stress.

SAME INCIDENT. BETTER COMMUNICATION.
STRONGER TEAMS.

Learn
Respond
Grow
with
VERIQTA



@veriqta



veriqta



veriqta



veriqta



@veriqta



@veriqta

Engineers
Communicate.
Systems
Recover.

BUILDING AN INCIDENT TIMELINE

Every Minute Matters

Capture the facts. Understand the sequence. Learn and improve.

1 KEY TIMELINE EVENTS



2 WHAT EACH EVENT MEANS

EVENT	DESCRIPTION
Alert time	When the alert was triggered.
Detection time	When the issue was first noticed (by monitoring or a person).
Acknowledgement	When someone accepted the alert.
Declaration	When the incident was officially declared (e.g. SEV-1, SEV-2).
Important observations	Key findings from logs, metrics, and checks.
Changes made	All actions taken (commands, config changes, rollbacks, etc.).
Mitigation	Steps taken to reduce or stop the impact.
Recovery	When the service returned to normal.
Resolution	When the incident was closed.

3 WHY TIMESTAMPS MATTER

- Help you understand the sequence of events
- Show how long each stage took
- Identify delays and bottlenecks
- Improve future response time
- Provide evidence for post-incident reviews
- Required for compliance and audits
- Help measure MTTA (Mean Time to Acknowledge) and MTTR (Mean Time to Recover)

4 EVIDENCE SOURCES

- Monitoring alerts (e.g. Prometheus, Datadog, CloudWatch)
- Logs (application, system, infrastructure)
- Traces (e.g. OpenTelemetry)
- Chat and incident channel messages (Slack, Teams)
- Deployment and change history (Git, CI/CD)
- Infrastructure events (Kubernetes, AWS, etc.)
- User reports and support tickets
- Screenshots and graphs

5 TIMELINE vs NARRATIVE

TIMELINE	NARRATIVE
✓ Chronological, time-based ✓ Focuses on facts ✓ Short and clear ✓ Used during and after the incident ✓ Helps with analysis and metrics	✓ Story-style description ✓ Explains what happened ✓ Includes context and details ✓ Used in post-incident reviews ✓ Helps identify root cause and lessons learned

7 JUNIOR ENGINEER TIPS

- Always use UTC for timestamps.
- Be accurate and avoid guessing.
- Record events as they happen.
- Include both successes and failures.
- Use the timeline during the incident, not just afterwards.
- A good timeline makes the post-incident review easier.

6 EXAMPLE INCIDENT TIMELINE

TIME (UTC)	EVENT	DETAILS
14:02	Alert triggered	High error rate on API
14:03	Detected	On-call engineer noticed alert
14:04	Acknowledged	Alert acknowledged in PagerDuty
14:06	Declared	SEV-1 incident declared
14:08	Observation	New deployment 10 minutes ago
14:12	Change made	Rolled back to previous version
14:15	Mitigation	Error rate started to drop
14:20	Recovery	Service restored, metrics normal
14:25	Resolution	Incident closed
14:30	Follow-up	Post-incident review scheduled

Learn
Improve
Be Better
with
VERIQTA

A CLEAR TIMELINE
BUILDS A STRONGER, MORE RELIABLE SYSTEM.

Incidents
Teach
Resilient
Engineers

ESCALATION, ON-CALL, AND MAJOR INCIDENTS

Stronger Engineers Support Each Other

Know when to escalate. Get the right people involved. Resolve faster.

1 WHEN TO ESCALATE



- Customer impact is high or increasing
- You cannot resolve the issue within expected time
- The issue is outside your expertise
- Multiple services are affected
- You suspect a security, data, or compliance issue
- You need approval for high-risk changes

ESCALATE EARLY. IT IS BETTER TO ASK FOR HELP THAN TO MAKE IT WORSE.

2 ON-CALL RESPONSIBILITIES



- Monitor alerts and respond quickly
- Acknowledge and triage alerts
- Investigate and identify impact
- Communicate with the team
- Take ownership or escalate
- Document the incident
- Stay available until handover
- Follow runbooks and procedures

ON-CALL MEANS RESPONSIBILITY, NOT PERFECTION.

3 PAGING ANOTHER ENGINEER



- Use the correct on-call rotation tool (e.g. PagerDuty, Opsgenie)
- Provide a clear summary of the issue
- Include impact and urgency
- Avoid unnecessary pages
- Follow escalation policies
- Stay available for a handover
- Confirm the engineer received the page

BE CLEAR, CONCISE, AND FACTUAL WHEN YOU PAGE SOMEONE.

4 SERVICE OWNERSHIP



- Every service should have an owner
- Service owners know the system best
- They are responsible for response and resolution
- They maintain runbooks and documentation
- They coordinate with other teams
- Ownership should be clear and up to date

CLEAR OWNERSHIP REDUCES CONFUSION AND SPEEDS UP RECOVERY.

5 BRINGING IN SPECIALISTS



- Some incidents require specialist skills (e.g. database, network, security, cloud)
- Involve specialists early when needed
- Provide context and what you have tried
- Be specific about the help you need
- Keep communication in the incident channel
- Respect everyone's time

THE RIGHT EXPERT AT THE RIGHT TIME CAN SAVE HOURS.

6 MANAGEMENT ESCALATION



- Escalate to management for major incidents (e.g. SEV-1)
- Keep management informed of impact, progress, and risks
- Be honest about what is known and unknown
- Request support when needed (e.g. customer communication, resources)
- Follow your company's escalation policy

MANAGEMENT ESCALATION ENSURES THE RIGHT SUPPORT AND DECISIONS.

7 SECURITY ESCALATION



- Escalate immediately for suspected security incidents
- Involve the security team (e.g. SOC, CSIRT)
- Do not attempt to handle security incidents alone
- Preserve evidence and avoid making changes
- Follow the security incident process
- Be aware of compliance and reporting requirements

WHEN IN DOUBT, ESCALATE TO SECURITY IMMEDIATELY.

8 THIRD-PARTY / VENDOR INCIDENTS



- Identify if the issue is with a third-party service (e.g. AWS, Azure, GitHub, Stripe)
- Check vendor status pages
- Open a support ticket with the vendor
- Share information with your team and stakeholders
- Monitor for updates
- Consider workarounds or failover options
- Keep the incident timeline updated

EXTERNAL INCIDENTS STILL IMPACT YOUR CUSTOMERS. COMMUNICATE CLEARLY.

9 INCIDENT HANDOVERS



- Provide a clear summary of the current state
- Share what has been done and what is pending
- Include important links, logs, and findings
- Communicate any risks or open questions
- Confirm the incoming engineer understands
- Stay available briefly for questions

A GOOD HANDOVER KEEPS MOMENTUM AND PREVENTS REPEAT WORK.

10 FATIGUE DURING LONG INCIDENTS



- Long incidents can be mentally and physically exhausting
- Take breaks when possible
- Rotate team members
- Stay hydrated and rest when you can
- Be aware of reduced decision-making ability
- Speak up if you need to step away

A TIRED ENGINEER MAKES MORE MISTAKES. TAKE CARE OF YOURSELF.

11 WHY ASKING FOR HELP EARLY IS A SKILL



- Shows good judgment and professionalism
- Reduces time to resolution
- Prevents small issues from becoming major incidents
- Leverages team knowledge and experience
- Builds trust and a stronger team
- It is not a sign of weakness, it is an engineering skill

ASKING FOR HELP EARLY IS A STRENGTH, NOT A WEAKNESS.

NO ENGINEER SOLVES MAJOR INCIDENTS ALONE. STRONG TEAMS RECOVER TOGETHER.

Better Incidents Build Better Engineers.



@veriqta



veriqta



veriqta



veriqta



@veriqta



@veriqta

Learn Share Grow with VERIQTA

RECOVERY IS NOT FINISHED UNTIL YOU VERIFY

Verify
Validate
Stabilize
Move
Forward

Service restored is progress. Verification ensures it stays that way.

1 SERVICE RESTORED vs INCIDENT RESOLVED



- Service restored means the system is working again.
- Incident resolved means you have confirmed stability, no recurrence, and all impact is over.
- Do not close the incident too early.

RESTORE THE SERVICE. THEN VERIFY. THEN RESOLVE.

2 VERIFY HEALTH CHECKS



- Check application and infrastructure health checks.
- Verify Kubernetes readiness and liveness probes.
- Ensure all endpoints return expected responses.
- Confirm no failing checks across all regions or environments.

HEALTH CHECKS SHOULD BE GREEN AND STABLE.

3 VERIFY ERROR RATE



- Check that error rate has returned to normal levels.
- Compare with pre-incident baseline.
- Look for any remaining spikes.
- Monitor for a sustained period, not just a few minutes.

A LOW AND STABLE ERROR RATE CONFIRMS RECOVERY.

4 VERIFY LATENCY



- Ensure response times are back to normal.
- Compare with historical baseline.
- Check key user journeys.
- Look for regional differences.

LATENCY SHOULD BE WITHIN NORMAL RANGE.

5 VERIFY TRAFFIC



- Confirm traffic has returned to expected levels.
- Check for any unexpected drops.
- Validate key services are receiving requests.
- Ensure load balancers and routing are working correctly.

TRAFFIC SHOULD BE NORMAL FOR YOUR ENVIRONMENT.

6 VERIFY DEPENDENCIES



- Check all dependent services and third-party APIs.
- Ensure databases, queues, caches, and external services are healthy.
- Verify there are no hidden failures in downstream systems.

YOUR SERVICE IS ONLY COVERED IF ITS DEPENDENCIES ARE ALSO HEALTHY.

7 CHECK LOGS



- Look for new errors or warnings.
- Confirm the original issue is resolved.
- Check application, system, and infrastructure logs.
- Look for repeated patterns or side effects.

CLEAN LOGS HELP CONFIRM A STABLE RECOVERY.

8 CHECK CUSTOMER-FACING BEHAVIOR



- Test the application from a user's perspective.
- Verify key user journeys (login, checkout, search, etc.).
- Check for any lingering issues.
- Monitor support channels for new complaints.

THE SERVICE MUST WORK FOR REAL USERS, NOT JUST ENGINEERS.

9 WATCH FOR RECURRENCE



- Monitor for a reasonable period after recovery.
- Look for repeated errors or unusual patterns.
- Keep dashboards and alerts active.
- Be alert to related issues that may appear later.

NO RECURRENCE MEANS A STABLE FIX.

10 REMOVE TEMPORARY MITIGATIONS CAREFULLY



- Identify all temporary changes made during the incident.
- Remove them one by one.
- Verify the service remains stable after each removal.
- Document what was removed and when.

DO NOT LEAVE TEMPORARY FIXES IN PLACE LONGER THAN NEEDED.

11 CONFIRM MONITORING HAS RETURNED TO NORMAL



- Ensure all alerts are back to normal.
- Check that dashboards show healthy metrics.
- Verify alert noise has stopped.
- Confirm SLOs are recovering.
- Keep monitoring closely for at least 24 hours.

MONITORING CONFIRMS THE SYSTEM IS TRULY STABLE.

12 RECOVERY CHECKLIST



- ✓ Service is working as expected
- ✓ Health checks are green
- ✓ Error rate is normal
- ✓ Latency is normal
- ✓ Traffic has returned to expected levels
- ✓ Dependencies are healthy
- ✓ Logs show no new errors
- ✓ Customer-facing behavior is verified
- ✓ No recurrence after monitoring period
- ✓ Temporary mitigations removed
- ✓ Monitoring and alerts are normal

Incidents
Teach
Resilience
VERIQTA

VERIFY TODAY.
A MORE RELIABLE TOMORROW.

Learn
Build
Grow
with
VERIQTA

POST-INCIDENT REVIEW AND LEARNING

Turn incidents into knowledge. Better systems. Fewer incidents.

Learn From Incidents Build Better Systems

1 WHAT HAPPENED?



- Summarize the incident in simple, clear terms.
- What were the symptoms?
- Which services were affected?
- What actually happened?

BE FACTUAL. AVOID ASSUMPTIONS.

2 WHAT WAS THE IMPACT?



- How many users were affected?
- What services were down or degraded?
- Was there a financial, operational, or reputational impact?
- How long did it last?

QUANTIFY THE IMPACT WHERE POSSIBLE.

3 HOW WAS IT DETECTED?



- Monitoring alert?
- Customer report?
- Internal report?
- Synthetic check?
- Third-party notification?

UNDERSTAND IF DETECTION WAS GOOD ENOUGH.

4 WHAT WAS THE TIMELINE?



- When did it start?
- When was it detected?
- When was it acknowledged?
- When was it resolved?
- Key events and timestamps.

A CLEAR TIMELINE HELPS FIND GAPS AND IMPROVE RESPONSE TIME.

5 WHAT CONTRIBUTED TO THE FAILURE?



- Technical issues
- Configuration changes
- Infrastructure problems
- Dependency failures
- Process gaps
- Human factors
- External factors

INCIDENTS RARELY HAVE A SINGLE CAUSE.

6 WHAT HELPED RECOVERY?



- What worked well?
- Which tools and processes were useful?
- What did the team do well?
- Was there good communication?
- Did runbooks help?

CAPTURE WHAT WORKED SO IT CAN BE REUSED.

7 WHAT MADE RECOVERY HARDER?



- Missing documentation
- Unclear ownership
- Lack of visibility
- Slow approvals
- Limited access
- Insufficient tools
- Complex architecture
- External dependencies

IDENTIFY FRICTION POINTS AND REMOVE THEM.

8 ROOT CAUSE AND CONTRIBUTING FACTORS



- Identify the root cause.
- List contributing factors.
- Look at technical, process, and people factors.
- Avoid stopping at "human error."

ROOT CAUSE IS A SYSTEM PROBLEM, NOT A PERSON.

9 FIVE WHYS, USED CAREFULLY



- Ask "why" repeatedly to find the underlying cause.
- Look beyond the obvious.
- Stop when you reach a meaningful system-level cause.
- Do not use it to blame individuals.

USE FIVE WHYS TO UNDERSTAND, NOT TO ASSIGN BLAME.

10 BLAMELESS ANALYSIS



- Focus on the system, not individuals.
- Understand how and why the incident happened.
- Encourage honest discussions.
- Create a safe environment for learning.
- Blameless does not mean no accountability.

A BLAMELESS CULTURE LEADS TO STRONGER SYSTEMS.

11 CORRECTIVE ACTIONS



- Define clear action items.
- Assign owners.
- Set realistic deadlines.
- Track progress.
- Verify completion.

TURN LESSONS INTO ACTIONS.

12 IMPROVING SYSTEMS



- Improve alerts and monitoring.
- Update runbooks.
- Add automation.
- Strengthen testing.
- Simplify architecture where possible.
- Reduce single points of failure.

MOVE FROM REACTIVE TO PROACTIVE.

13 WHY "HUMAN ERROR" IS RARELY USEFUL



- "Human error" does not explain why the system allowed the error.
- Look at training, tools, processes, and system design.
- People make mistakes. Good systems reduce the impact.
- Focus on how to prevent it, not who to blame.

FIX THE CONDITIONS, NOT JUST THE PEOPLE.

14 POST-INCIDENT REVIEW CHECKLIST



- ✓ All key questions answered
- ✓ Timeline completed
- ✓ Root cause and contributing factors identified
- ✓ Action items defined with owners and deadlines
- ✓ Monitoring and runbooks updated
- ✓ Lessons shared with the wider team
- ✓ Follow-up review scheduled

SAME INCIDENTS SHOULD NOT HAPPEN AGAIN.

Learn Improve Grow with VERIQTA

INCIDENTS ARE INEVITABLE REPEATING THEM IS OPTIONAL.

Better Systems Happier Engineers

FINAL PRODUCTION INCIDENT LAB

DETECT, RESPOND, RECOVER

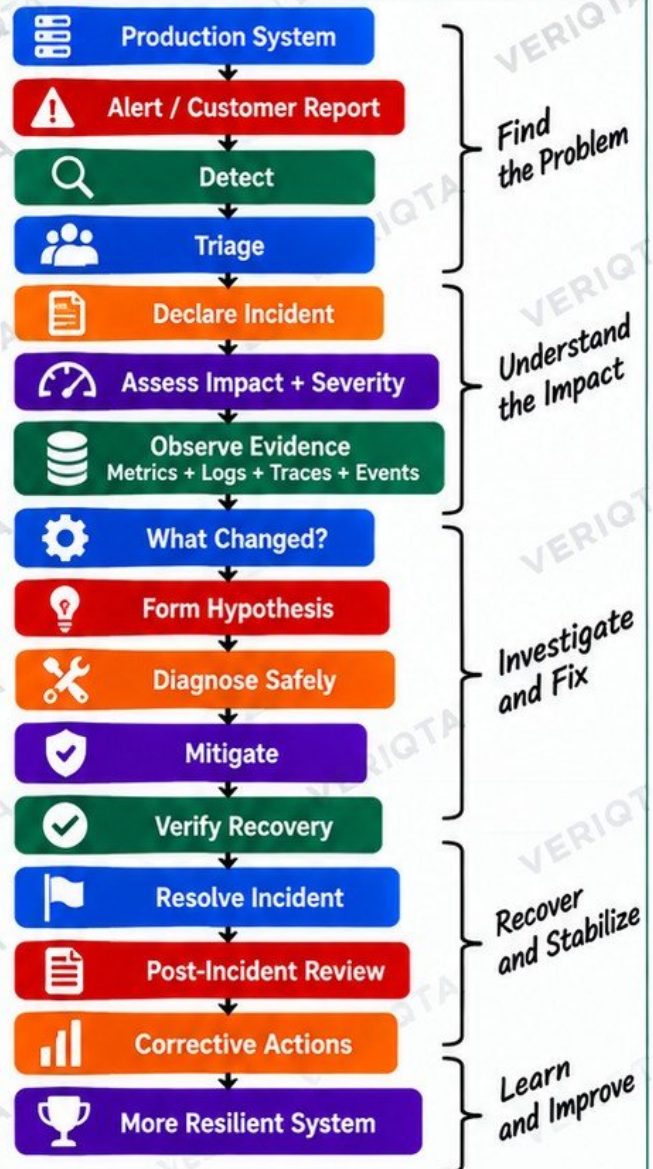
A hands-on scenario to apply everything you have learned.

SAME
PROCESS.
STRONGER
SYSTEMS.

LAB STEPS

- 1  **Application is healthy**
Start with a working application in Kubernetes.
- 2  **Deploy a deliberately broken release**
Use a bad image tag or faulty configuration.
- 3  **Monitoring detects rising errors**
Observe alerts, metrics and dashboards.
- 4  **Acknowledge the alert**
Confirm you have seen the alert.
- 5  **Confirm customer impact**
Check if users are affected.
- 6  **Declare the incident**
Open the incident and notify the team.
- 7  **Assign severity**
Use SEV levels based on impact.
- 8  **Start the incident timeline**
Record key events with timestamps.
- 9  **Inspect metrics and logs**
Identify symptoms and patterns.
- 10  **Check Kubernetes workloads**
Investigate pods, deployments and events.
- 11  **Identify the recent deployment**
Check what changed and when.
- 12  **Form and test a hypothesis**
Validate with real evidence.
- 13  **Choose mitigation**
Rollback, scale, or apply a fix.
- 14  **Roll back the bad release**
Use kubectl or your deployment tool.
- 15  **Verify application recovery**
Check health, metrics and user access.
- 16  **Monitor for recurrence**
Keep watching for a reasonable period.
- 17  **Resolve the incident**
Once the service is stable.
- 18  **Conduct a short post-incident review**
Discuss what happened and what to improve.
- 19  **Create corrective actions**
Assign owners and deadlines.

FINAL INCIDENT RESPONSE MENTAL MODEL



KEY TAKEAWAYS

- Follow a structured process, even under pressure.
- Use evidence, not assumptions.
- Communicate clearly and early.
- Verify before closing.
- Every incident is a learning opportunity.

REAL ENGINEERS
DON'T JUST FIX INCIDENTS.
THEY MAKE SURE THEY
HAPPEN LESS OFTEN."

Better
Engineers
Brighter
Systems